



UNIT 1

Fundamentals of Artificial Intelligence

AI Representation • Knowledge & Search • Intelligent Agents • Problem Formulation

What We'll Cover

Two major themes across this unit

PART A — AI Foundations

- Introduction to AI & Representation
- Non-AI vs. AI Techniques
- Representation of Knowledge
- Knowledge Base Systems
- State Space Search
- Production Systems
- Problem Characteristics
- Turing Test

PART B — Intelligent Agents

- Agents & Environments
- Concept of Rationality
- Nature of Environments
- Structure of Agents
- Types of Agents
- Problem-Solving Agents
- Problem Formulation
- Vacuum World, 8-Queens, Route Finding, Robot Navigation

Unit-1 Part: A

AI Foundations

What is Artificial Intelligence?

The science and engineering of making machines that can perform tasks which, when done by humans, require intelligence — reasoning, learning, perception, and decision-making.

Thinking Humanly

Systems that model human cognitive processes (cognitive modeling).

Thinking Rationally

Systems that follow laws of logic to reach correct conclusions.

Acting Humanly

Systems that behave indistinguishably from humans (e.g., Turing Test).

Acting Rationally

Systems that act to achieve the best expected outcome (rational agents).

Non-AI vs. AI Techniques

Non-AI vs. AI Techniques

Why intelligent problem solving needs more than brute force



A technique qualifies as an 'AI technique' when it organizes and uses domain knowledge efficiently, generalizes across problems, and can be understood and modified by people.

Non-AI Technique

Relies on exhaustive search / fixed algorithms

Limited scalability; ignores domain knowledge

AI Technique

Uses heuristics & structured knowledge representation

Efficient, flexible, handles uncertainty

Aspect	Non-AI Techniques (Conventional Programming)	AI Techniques
Definition	Uses predefined rules and algorithms to solve problems.	Mimics human intelligence by learning, reasoning, and making decisions.
Approach	Rule-based and deterministic.	Data-driven and adaptive.
Learning Ability	Cannot learn from experience; behavior remains fixed.	Learns from data and improves performance over time.
Decision Making	Decisions are based on explicit programming.	Decisions are based on learned patterns, inference, or search techniques.
Handling Uncertainty	Poor at handling incomplete or uncertain information.	Can handle uncertainty using probabilistic or heuristic methods.
Adaptability	Requires manual updates to modify behavior.	Adapts automatically as more data becomes available.
Problem Solving	Best for structured, well-defined problems.	Suitable for complex, dynamic, and uncertain problems.
Examples	Sorting algorithms, calculators, database queries, rule-based systems.	Game playing, speech recognition, image classification, recommendation systems, autonomous vehicles.
Development	Programmer defines every rule and condition.	Model learns rules from training data or intelligent search.
Performance	Predictable and consistent for predefined tasks.	Improves with training but may produce probabilistic results.

Example: Tic-Tac-Toe

Non-AI Technique	AI Technique
Player makes moves using predefined logic or random selection.	Computer uses intelligent algorithms such as Minimax, Alpha-Beta Pruning, or Reinforcement Learning to choose the optimal move.
No learning from previous games.	Can evaluate board states and optimize strategy.
Easy to implement but less challenging.	More complex but provides intelligent gameplay.

AI Techniques for Tic-Tac-Toe

1. Minimax Algorithm (Most Common) ★ ★ ★ ★ ★

- Concept: The AI assumes that both players play optimally. It explores all possible future moves and selects the move that maximizes its chance of winning while minimizing the opponent's chance.

Working:

- Generate all possible moves.
- Simulate each move recursively.

Assign scores:

- +1 → AI wins
- 0 → Draw
- -1 → Opponent wins
- Choose the move with the highest score.

Advantages

- Always plays optimally.
- Never loses (against perfect play).
- Simple to implement for small games.

Disadvantages

- Computationally expensive for larger games.
- Searches the complete game tree.

AI Techniques for Tic-Tac-Toe

2. Alpha-Beta Pruning ★ ★ ★ ★ ★

Concept: An optimized version of Minimax that ignores branches that cannot affect the final decision.

Working

Uses two values:

Alpha (α): Best score for the maximizing player.

Beta (β): Best score for the minimizing player.

Stops searching when $\alpha \geq \beta$.

Advantages

Faster than Minimax.

Produces the same optimal result.

Reduces unnecessary computations.

Disadvantages

Slightly more complex to implement.

AI Techniques for Tic-Tac-Toe

3. Heuristic-Based AI ★ ★ ★

Concept: Instead of searching every possible move, the AI evaluates the current board using rules.

Example Heuristics

Win if possible.

Block opponent's winning move.

Take the center.

Take a corner.

Take any remaining side.

Advantages

Very fast.

Easy to implement.

Disadvantages

Not always optimal.

Can be defeated.

AI Techniques for Tic-Tac-Toe

4. Rule-Based Expert System ★ ★ ★

Concept: Uses predefined IF–THEN rules.

Example Rules

If AI can win → Play winning move.

Else if opponent can win → Block.

Else if center is free → Take center.

Else choose a corner.

Else choose any empty cell.

Advantages

Simple.

Predictable.

Disadvantages

No learning capability.

Limited intelligence.

AI Techniques for Tic-Tac-Toe

5. Reinforcement Learning ★ ★ ★ ★

Concept: The AI learns by playing thousands of games and receiving rewards.

Reward Example

Win → +10

Draw → +1

Lose → -10

Popular methods:

Q-Learning

Deep Q-Network (DQN)

Advantages

Learns from experience.

Can improve over time.

Disadvantages

Requires many training games.

More complex than Minimax.

Advantages and Disadvantages of Non-AI Techniques

Advantages of Non-AI Techniques

- Simple to design and implement.
- Faster execution for predefined tasks.
- Predictable and easy to debug.
- Requires less computational power.

Disadvantages of Non-AI Techniques

- Cannot learn or improve.
- Performs poorly in dynamic or uncertain environments.
- Requires manual modification for new scenarios.

Advantages and Disadvantages of AI Techniques

Advantages of AI Techniques

- Learns and improves from experience.
- Handles complex decision-making problems.
- Can adapt to changing environments.
- Produces intelligent and optimal decisions.

Disadvantages of AI Techniques

- Requires more computational resources.
- Development is more complex.
- Often requires large amounts of training data.
- Results may not always be fully explainable.

Comparison in a Two-Player Game (Tic-Tac-Toe)

Feature	Non-AI Version	AI Version
Opponent Strategy	Random or fixed moves	Intelligent (Minimax/Alpha-Beta)
Difficulty Level	Easy	Medium to Impossible
Learning	No	Possible (with ML/RL)
Win Probability	Easily defeated	Difficult to defeat
Decision Time	Very fast	Slightly higher due to search

Representation of Knowledge

Knowledge Representation

- Knowledge Representation (KR) in AI focuses on how machines store and organize real-world information so they can reason, learn, and make intelligent decisions like humans.
- Represents knowledge in a structured form that computers can process.
- Helps AI systems perform reasoning and problem-solving tasks.
- Enables intelligent applications like medical diagnosis and language understanding.
- Allows machines to use stored knowledge and past experiences effectively.

Knowledge representation in AI systems



Types of Knowledge

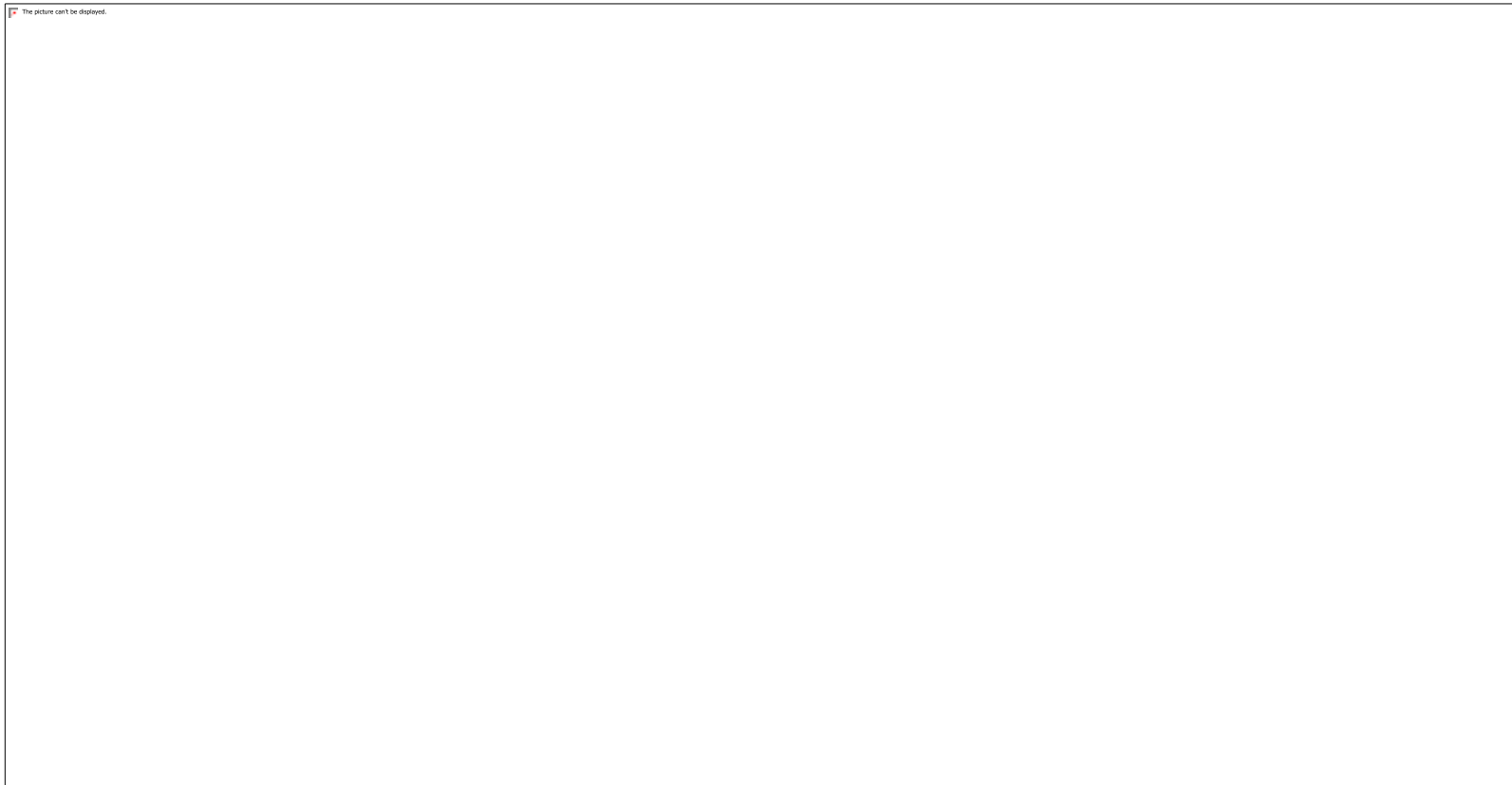
- **Declarative Knowledge:** Knowledge about facts and concepts it answers what something is . Example: Paris is the capital of France.
- **Procedural Knowledge:** Knowledge about how to perform a task or solve a problem. Example: Steps to sort numbers using an algorithm.
- **Meta knowledge:** Knowledge about other knowledge or how knowledge is used. Example: Knowing that a certain rule works better for solving math problems.
- **Heuristic Knowledge:** Experience based knowledge or rules of thumb used by experts. Example: A doctor using past experience to guess a possible disease.
- **Structural Knowledge:** Knowledge that shows relationships between concepts. Example: A car is a type of vehicle.

Types of knowledge



Relation between Knowledge and Intelligence

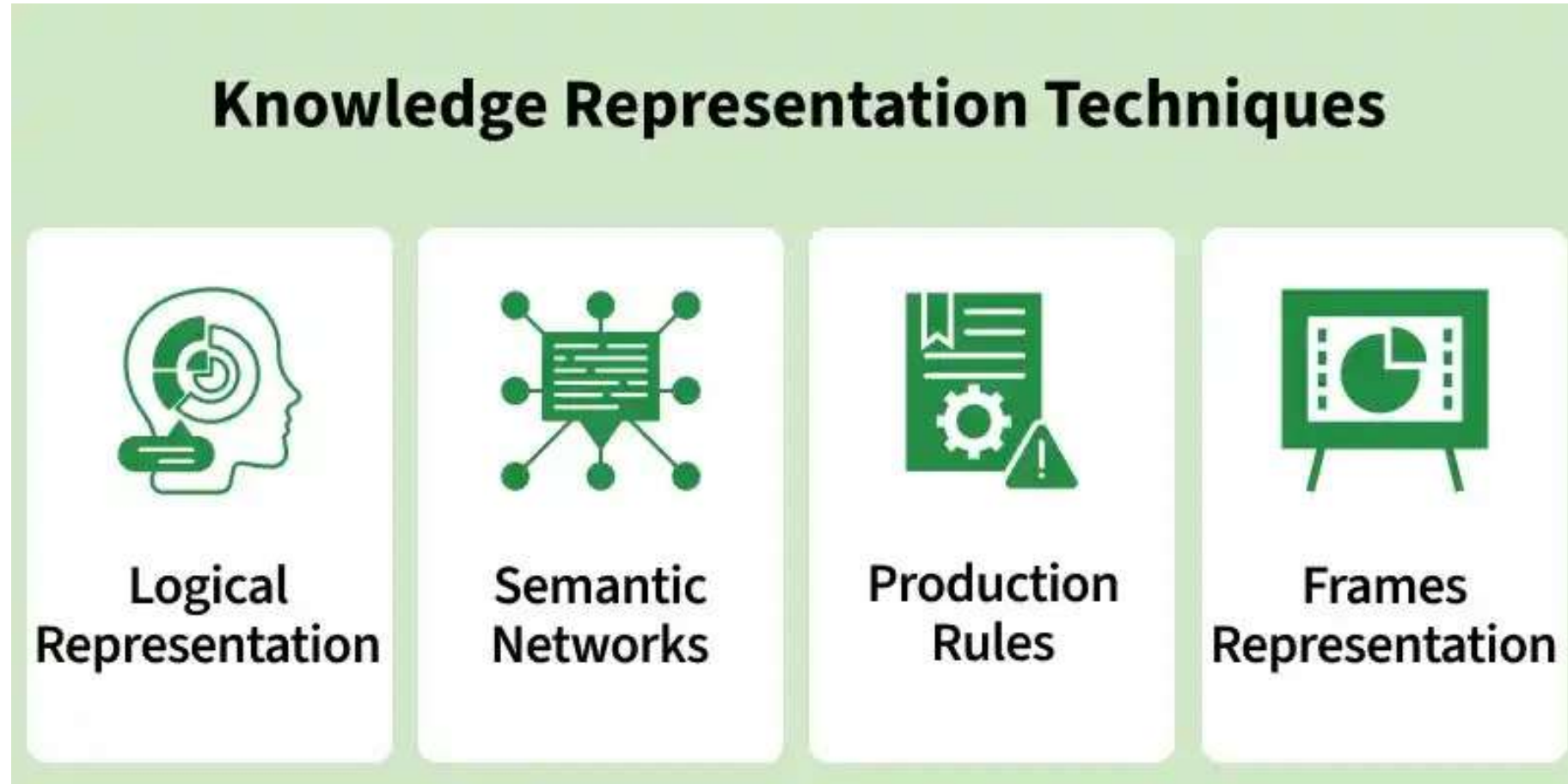
- Knowledge is essential for intelligence in both humans and AI systems. AI agents use knowledge and past experience to understand situations, make decisions, and respond intelligently.
- The diagram shows that intelligent behavior is possible only when a decision-maker uses knowledge from the environment.



Relation between Knowledge and Intelligence

- This relation can be understood as follows:
- Knowledge about the real world is essential for building intelligent AI systems
- AI agents use knowledge to understand inputs and take appropriate actions.
- An agent can respond correctly only if it has knowledge or prior experience.
- Without knowledge, an AI system cannot display intelligent behaviour.

Techniques of knowledge representation



1. Logical Representation

- Logical representation uses formal rules and logic to represent knowledge in AI and helps systems draw conclusions based on given conditions. It includes the following ideas:
- **Syntax:** Rules that decide how symbols and sentences are written in logic.
- **Semantics:** Rules that give meaning to logical sentences.
- Logical representation mainly includes Propositional Logic and Predicate Logic.
- **Advantages**
 - Helps in logical reasoning and decision making.
 - Forms the basis of many programming languages.
- **Disadvantages**
 - Can be difficult to use and understand.
 - Reasoning using logic can sometimes be slow or inefficient.

2. Semantic Network Representation

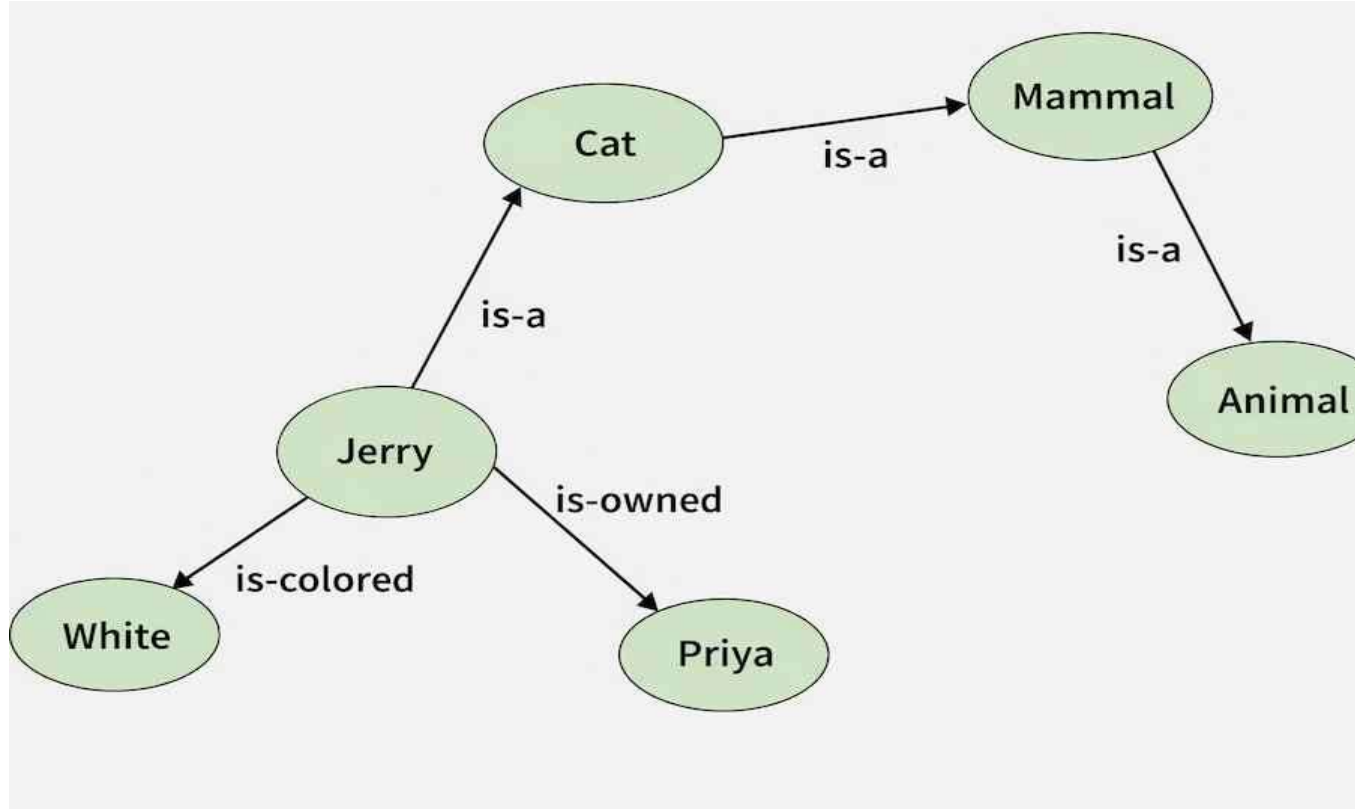
- A semantic network represents knowledge using a graph structure with nodes (concepts or objects) and arcs (relationships). It helps show connections between different concepts.
- IS-A relation: Shows inheritance (e.g. Cat is a Mammal).
- Kind-of relation: Shows category relationships.

Advantages

- Natural and easy way to represent knowledge.
- Simple to understand and visualize.
- Can be easily expanded.

Disadvantages

- Searching large networks can take more time.
- Difficult to create very large networks like human memory.
- Lacks clear standards for link names and logical expressions.



3. Frame Representation

- A frame is a data structure used to represent knowledge about an object or situation using slots (attributes) and values. It organizes related information together and is widely used in NLP and computer vision.
- Example: A frame for a Book may include slots such as title, author, year and pages.
- **Advantages**
 - Groups related information together, making programming easier.
 - Flexible and easy to extend by adding new attributes.
 - Easy to understand and visualize.
- **Disadvantages**
 - Inference and reasoning are not always efficient.
 - Sometimes too generalized for complex reasoning.

4. Production Rules

- Production rules represent knowledge in the form of IF–THEN rules. When a condition is satisfied, the system performs the corresponding action. A production rule system has three main parts:
 - Set of rules
 - Working memory (current problem state)
 - Recognize act cycle (process of checking conditions and applying rules)
 - Example: IF (bus arrives) THEN (get into the bus).
- **Advantages**
 - Easy to understand because rules are written in simple language.
 - Highly modular; rules can be added, removed, or modified easily.
- **Disadvantages**
 - Systems usually do not learn from past results.
 - Many active rules can make the system slower or inefficient.

Applications

- Used in expert systems to provide advice and support decision-making in specialized fields such as medical diagnosis.
- Helps NLP systems understand and generate human language in applications like chatbots and translation tools.
- Enables robots to understand their environment and perform tasks such as navigation and obstacle avoidance.
- Supports the semantic web by helping machines organize and understand relationships between web information.
- Assists cognitive computing systems in analyzing large datasets to generate insights in areas like healthcare research.

State Space Search in AI

State Space Search

State Space Search is used to solve problems by finding different possible states and their transitions. In simple terms it's like finding the best route to goal by trying different paths.

- It works by systematically checking each state and moving to new states until the goal is reached. It consider the problem as a series of steps and it moves from one state to another until the goal is reached.
- This method can be applied to various AI tasks such as pathfinding, puzzle solving, game playing and more.

The core idea behind state space search is to think of the problem as a graph where:

- Nodes represent different states of the problem.
- Edges represent transitions or actions that change one state into another.

Terminologies consist of:

- **State:** A specific configuration of the problem.
- **Initial State:** Starting point of the search.
- **Goal State:** Desired end configuration.
- **Transition:** An action that changes one state to another.
- **Path:** A sequence of states connected by transitions.
- **Search Strategy:** Method used to explore the state space.

Principles and Features of State Space Search

The efficiency of state space search depends on several key factors. Understanding these helps in choosing right strategy and optimizing the search process.

1. **Expansiveness:** Number of new states a given state can generate. More broader problems require finding more possibilities which increases search complexity.
2. **Branching Factor:** Average number of successors per state. A higher branching factor makes search tree wider which increases time and resources required.
3. **Depth:** Number of steps from the initial state to the goal state. Deeper trees increase the search time as more states must be explored.
4. **Completeness:** A search is complete if it guarantees finding a solution if one exists. It ensures the algorithm will eventually reach the goal.
5. **Optimality:** A search is optimal if it guarantees finding the best solution, according to a specific criterion liked least cost or shortest path.
6. **Time Complexity:** The time required for the search which get affected by branching factor and depth. More expansive or deeper trees increase the time needed.
7. **Space Complexity:** Memory required for the search. It depends on how many states need to be stored at once. Larger branching factors or deeper searches use more memory.

Steps in State Space Search

- **Step 1: Define the State Space**

- Identify all possible states and their transitions. To do this model the problem in a way that includes all relevant configurations and actions.

- **Step 2: Pick a Search Strategy**

Choose a method for exploring the state space. Common strategies include:

- Breadth-First Search (BFS): Explores all nodes at one depth level before moving to the next, ideal for unweighted graphs.
- Depth-First Search (DFS): Explores a branch as deeply as possible before backtracking. It uses less memory but may not guarantee completeness or optimality.
- Uniform Cost Search (UCS): Expands the least costly node first which ensures the lowest-cost solution.
- Greedy Best-First Search: Expands nodes that appear closest to the goal based on a heuristic.
- A* Search Algorithm: Combines path cost and heuristic to guarantee completeness and optimality.

- **Step 3: Start the Search**

- Add initial state and begin the search from there.

Steps in State Space Search

- Step 4: Extend the Nodes
 - Using the selected search technique it expands node from start state and helps in generating successor states and adding them to the frontier. If a state matches the goal it retrace path to the solution and stop the search.
- Step 5: Address State Repetition
 - Prevent revisiting same state by tracking visited states helps in avoiding cycles and unnecessary exploration.
- Step 6: End the Search
 - The search ends when goal state is found or when all states have been explored without finding a solution.

Applications of State Space Search

State space search is used in many different fields such as:

1. Pathfinding: Finding the best pathways using algorithms such as A* in robotics and GPS.
2. Puzzle solving: Resolving puzzles like Rubik's Cube, Sudoku and the 8-puzzle.
3. AI for gaming: To identify good moves in board games like chess, checkers and others.
4. Planning: Automated scheduling of tasks in logistics and robotics to achieve a specific objective.
5. Natural language processing: It involves computer translation and sentence parsing by examining many interpretations.

Challenges in State Space Search

1. Complexity: High branching factors can cause an exponential growth in the number of states to be explored.
 2. Resource Limitations: Memory and processing power limit size of the state space that can be practically searched.
 3. Quality of Heuristics: The effectiveness of the search is limited by the quality of the heuristic function.
- State space search remains a fundamental technique in AI which helps systems to solve complex problems by systematically exploring various possibilities to reach optimal solutions.

Production System in AI

Production System in AI

- In artificial intelligence (AI), a production system refers to a type of rule-based system that is designed to provide a structured approach to problem solving and decision-making. This framework is particularly influential in the realm of expert systems, where it simulates human decision-making processes using a set of predefined rules and facts.
- **Example** - Expert System for Medical Diagnosis.
- **Scenario:** A patient comes to a healthcare facility with the following symptoms: fever, severe headache, sensitivity to light, and stiff neck.
- Medical diagnosis operates in the following manner:
- **Input:** A healthcare professional inputs the symptoms into MediDiagnose.
- **Processing:**
 - MediDiagnose reviews its knowledge base for rules that match the given symptoms.
 - It identifies several potential conditions but recognizes a strong match for meningitis based on the combination of symptoms.
- **Output:**
 - The system suggests that meningitis could be a possible diagnosis and recommends further tests to confirm, such as a lumbar puncture.
 - It also provides a list of other less likely conditions based on the symptoms for comprehensive differential diagnosis.
 - MediDiagnose uses its rule-based system to quickly filter through vast amounts of medical data to provide preliminary diagnoses. This assists doctors in focusing their investigative efforts more efficiently and potentially speeds up the process of reaching an accurate diagnosis.

Key Components of a Production System in AI

- The key components of production system includes:
 1. **Knowledge Base:** This is the core repository where all the rules and facts are stored. In AI, the knowledge base is critical as it contains the domain-specific information and the if-then rules that dictate how decisions are made or actions are taken.
 2. **Inference Engine:** The inference engine is the mechanism that applies the rules to the known facts to derive new facts or to make decisions. It scans the rules and decides which ones are applicable based on the current facts in the working memory. It can operate in two modes:
 - **Forward Chaining (Data-driven):** This method starts with the available data and uses the inference rules to extract more data until a goal is reached.
 - **Backward Chaining (Goal-driven):** This approach starts with a list of goals and works backwards to determine what data is required to achieve those goals.
 3. **Working Memory:** Sometimes referred to as the fact list, working memory holds the dynamic information that changes as the system operates. It represents the current state of knowledge, including facts that are initially known and those that are deduced throughout the operation of the system.
 4. **Control Mechanism:** This governs the order in which rules are applied by the inference engine and manages the flow of the process. It ensures that the system responds appropriately to changes in the working memory and applies rules effectively to reach conclusions or solutions.

Types of Production Systems

1. Rule-Based Systems

1. Explanation of Rule-Based Reasoning

- Rule-based systems operate by applying a set of pre-defined rules to the given data to deduce new information or make decisions. These rules are generally in the form of conditional statements (if-then statements) that link conditions with actions or outcomes.

2. Examples of Rule-Based Systems in AI

- Diagnostic Systems: Like medical diagnosis systems that infer diseases from symptoms.
- Fraud Detection Systems: Used in banking and insurance, these systems analyze transaction patterns to identify potentially fraudulent activities.

Types of Production Systems

2. Procedural Systems

1. Description of Procedural Knowledge
 - Procedural systems utilize knowledge that describes how to perform specific tasks. This knowledge is procedural in nature, meaning it focuses on the steps or procedures required to achieve certain goals or results.
2. Applications of Procedural Systems
 - Manufacturing Control Systems: Automate production processes by detailing step-by-step procedures to assemble parts or manage supply chains.
 - Interactive Voice Response (IVR) Systems: Guide users through a series of steps to resolve issues or provide information, commonly used in customer service.

Types of Production Systems

3. Declarative Systems

1. Understanding Declarative Knowledge

- Declarative systems are based on facts and information about what something is, rather than how to do something. These systems store knowledge that can be queried to make decisions or solve problems.

2. Instances of Declarative Systems in AI

- Knowledge Bases in AI Assistants: Power virtual assistants like Siri or Alexa, which retrieve information based on user queries.
- Configuration Systems: Used in product customization, where the system decides on product specifications based on user preferences and declarative rules about product options.

Each type of production system offers different strengths and is suitable for various applications, from straightforward rule-based decision-making to complex systems requiring intricate procedural or declarative reasoning.

How Production Systems Function? & Applications

The operation of a production system in AI follows a cyclic pattern:

Match: The inference engine checks which rules are triggered based on the current facts in the working memory.

- Select: From the triggered rules, the system (often through the control mechanism) selects one based on a set of criteria, such as specificity, recency, or priority.
- Execute: The selected rule is executed, which typically modifies the facts in the working memory, either by adding new facts, changing existing ones, or removing some.

Applications of Production Systems in AI

Production systems are used across various domains where decision-making can be encapsulated into clear, logical rules:

- Expert Systems: For diagnosing medical conditions, offering financial advice, or making environmental assessments.
- Automated Planning: Used in logistics to optimize routes and schedules based on current data and objectives.
- Game AI: Manages non-player character behavior and decision-making in complex game environments.

Characteristics of Artificial Intelligence Problems

Core characteristics of Problems in Artificial Intelligence (AI)

- Problems in Artificial Intelligence (AI) come in different forms, each with its own set of challenges and potential for innovation. From image recognition to natural language processing, AI problems exhibit distinct characteristics that shape the strategies and techniques used to tackle them effectively.
- Let's explore the core characteristics that differentiate AI problems:
 1. **Learning and adaptation:**

AI systems should be capable of learning from data or experiences and adapting their behaviour accordingly. This enables them to improve performance over time and handle new situations more effectively.
 2. **Complexity:**

AI problems often involve dealing with complex systems or large amounts of data. AI systems must be able to handle this complexity efficiently to produce meaningful results.
 3. **Uncertainty:**

AI systems frequently operate in environments where outcomes are uncertain or incomplete information is available. They must be equipped to make decisions or predictions under such conditions.
 4. **Dynamism:**

Environments in which AI systems operate can change over time. These changes may occur unpredictably or according to specific rules, requiring AI systems to continually adjust their strategies or models.

Core characteristics of Problems in Artificial Intelligence (AI)

5. Interactivity:

Many AI applications involve interaction with users or other agents. Effective AI systems should be able to perceive, interpret, and respond to these interactions in a meaningful way.

6. Context dependence:

The behavior or performance of AI systems may depend on the context in which they operate. Understanding and appropriately responding to different contexts is essential for achieving desired outcomes.

7. Multi-disciplinary:

AI problems often require knowledge and techniques from multiple disciplines, including computer science, mathematics, statistics, psychology, and more. Integrating insights from these diverse fields is necessary for developing effective AI solutions.

8. Goal-oriented Design:

AI systems are typically designed to achieve specific objectives or goals. Designing AI systems with clear objectives in mind helps guide the development process and ensures that the resulting systems are focused on achieving meaningful outcomes.

Addressing the Challenges of AI Problems

The characteristics of AI problems present unique challenges that require innovative approaches to solution development. Some of the key aspects to consider in tackling these challenges include:

1. **Complexity and Uncertainty:** AI difficulties are sometimes characterized by highly variable domains that are difficult to predict exactly. Hence, AI algorithms should be installed with the skill of dealing with unclear circumstances and should make decisions that are based on imperfect data or noisy information.
2. **Algorithmic Efficiency:** Among the key challenges of this approach are the enormous search spaces, computational resources, and the efficiency of the algorithms in terms of problem-solving. Strategies like caching, pruning, and parallelization are among the most widely used implementations for better algorithmic speed.
3. **Domain Knowledge Integration:** Such numerous AI problems involve the ability to capture the rules and reasoning of the real world to model and solve the questions correctly. The AI machines that have been trained with expertise from relevant domains improve the accuracy and effectiveness of the applications in the real world.
4. **Scalability and Adaptability:** AI solutions should be able to process large datasets and complex cases at the same time, and they should also be versatile by responding to shifts in conditions and requirements. Strategies such as machine learning and reinforcement learning allow systems to do more than just perform according to the given tasks at hand; they empower systems to learn and progress over time.

Addressing the Challenges of AI Problems

- 5. Ethical and Social Implications:** AI technologies elicit ethical and social limitations concerning problems of bias, justice, privacy, and responsible office. Taking these implications into account, along with ethical frameworks, compliance frameworks, and stakeholder engagement, is essential. This approach will help position cryptocurrencies as a secure and trustworthy investment.
- 6. Interpretability and Explainability:** To achieve interpretability and explainability of AI algorithms for the sake of understanding and confidence among users and stakeholders, these algorithms should be knowable and comprehensible enough. Examples like chatbots producing natural-like conversation could better clarify the working scheme of AI technology.
- 7. Robustness and Resilience:** AI machinery should perform against its being hacked or affected by adversarial attacks, inaccuracies (errors), and environmental changes. Robustness testing, the construction of mechanisms for error handling, and the building up of redundancy must be taken seriously by AI systems to ensure their reliability and stability.
- 8. Human-AI Collaboration:** Successful human-AI entente is the key component to making the most of our advantages as well as artificial intelligence skills. Achieving AI solutions that are capable of supporting human skills and more importantly, preferences will reduce human efforts correspondingly and bring the best performance.

Turing Test in Artificial Intelligence

Turing Test in Artificial Intelligence

The Turing Test is a widely recognized benchmark for evaluating a machine's ability to demonstrate human-like intelligence. The core idea is simple: A human judge engages in a text-based conversation with both a human and a machine. The judge's task is to determine which participant is human and which is the machine. If the judge is unable to distinguish between the human and the machine based solely on the conversation, the machine is said to have passed the Turing Test.

Criteria for the Turing Test

- The Turing Test does not require the machine to be correct or logical in its responses but rather to be convincing in simulating human conversation. The test is fundamentally about deception—the machine must fool the judge into believing that it is human.
- The key criteria include the following for turing test:
 1. **Natural Language Processing (NLP):** The machine must understand and generate human language fluently.
 2. **Knowledge Representation:** The machine needs to handle and manipulate knowledge to provide contextually relevant responses.
 3. **Reasoning:** The machine should demonstrate some form of logical reasoning, even if flawed, to sustain a conversation.
 4. **Learning:** Ideally, the machine should learn from the interaction, adapting its responses over time.

How the Turing Test Works?

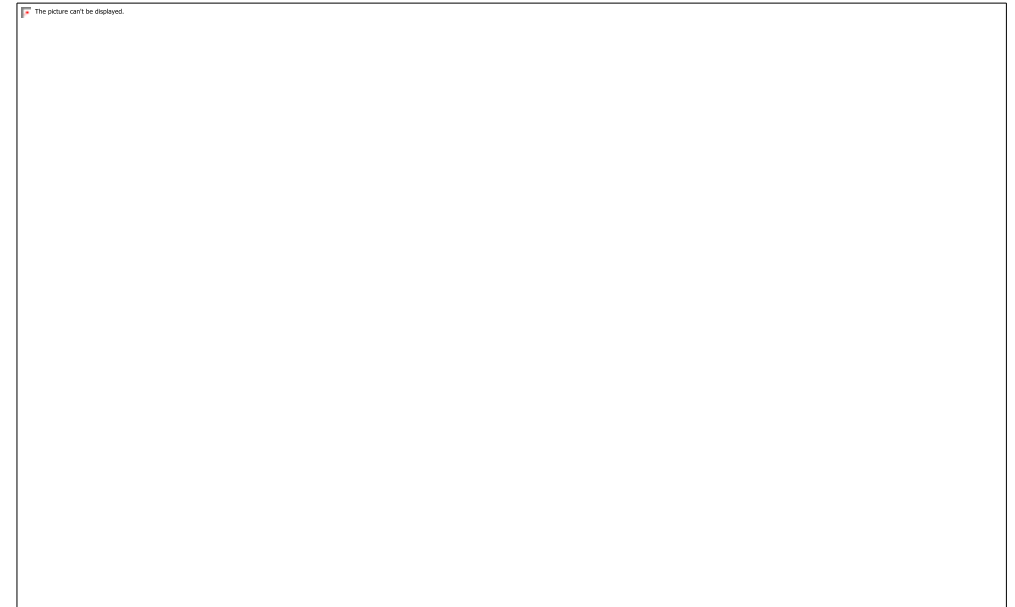
In a typical Turing Test scenario, three participants are involved: two humans and one machine.

The interrogator, a human judge, is isolated from the other two participants. The judge asks questions to both the human and the machine, aiming to identify which one is the human. The machine's goal is to respond in a way that makes it indistinguishable from the human participant. If the judge cannot reliably identify the machine, the machine is considered to have passed the Turing Test.

Here's an example of a conversation between the interrogator and the machine:

- Judge: Are you a computer?
- Machine: No.
- Judge: Multiply 158745887 by 56755647.
- Machine: (After a long pause) [Provides an incorrect answer].
- Judge: Add 5,478,012 and 4,563,145.
- Machine: (Pauses for 20 seconds and then responds) 10,041,157.

If the judge cannot distinguish between the responses of the human and the machine, the machine passes the test. The conversation is limited to a text-only format, such as a computer keyboard and screen, to prevent the judge from being influenced by any non-verbal cues.



Advantages-Disadvantages of the Turing Test

Advantages of the Turing Test in Artificial Intelligence

1. Evaluating Machine Intelligence: The Turing Test provides a simple and well-known method for assessing machine intelligence.
2. Setting a Benchmark: It establishes a benchmark for AI research and offers a goal for researchers to strive towards.
3. Inspiring Research: The Turing Test has inspired numerous studies and experiments aimed at developing machines that can pass the test, driving progress in AI.
4. Simple to Administer: The Turing Test is relatively easy to administer, requiring just a computer and a human judge.

Disadvantages of the Turing Test in Artificial Intelligence

1. Limited Scope: The Turing Test focuses primarily on language-based conversations and does not account for other aspects of intelligence, such as perception, problem-solving, and decision-making.
2. Human Bias: The results can be influenced by the biases and preferences of the human judge, making it difficult to obtain objective and reliable results.
3. Not Representative of Real-World AI: The Turing Test may not accurately represent the kind of intelligence that machines need to demonstrate in real-world applications.

Part - B

Agents and Environments, Rationality

What are Agents?

- An **agent** is anything that can perceive its environment through sensors and acts upon that environment through effectors.
 1. A **human agent** has sensory organs such as eyes, ears, nose, tongue and skin parallel to the sensors, and other organs such as hands, legs, mouth, for effectors.
 2. A **robotic agent** replaces cameras and infrared range finders for the sensors, and various motors and actuators for effectors.
 3. A **software agent** has encoded bit strings as its programs and actions.

Agent Terminology & Rationality

- Performance Measure of Agent – It is the criteria, which determines how successful an agent is.
- Behavior of Agent – It is the action that agent performs after any given sequence of percepts.
- Percept – It is agents perceptual inputs at a given instance.
- Percept Sequence – It is the history of all that an agent has perceived till date.
- Agent Function – It is a map from the precept sequence to an action.

Rationality

- Rationality is nothing but status of being reasonable, sensible, and having good sense of judgment.
- Rationality is concerned with expected actions and results depending upon what the agent has perceived. Performing actions with the aim of obtaining useful information is an important part of rationality.

What is Ideal Rational Agent?

An ideal rational agent is the one, which is capable of doing expected actions to maximize its performance measure, on the basis of –

- Its percept sequence
- Its built-in knowledge base

Rationality of an agent depends on the following –

- The performance measures, which determine the degree of success.
- Agents Percept Sequence till now.
- The agents prior knowledge about the environment.
- The actions that the agent can carry out.

A rational agent always performs right action, where the right action means the action that causes the agent to be most successful in the given percept sequence. The problem the agent solves is characterized by Performance Measure, Environment, Actuators, and Sensors (PEAS).

Structure of Intelligent Agents

Agents structure

Agents structure can be viewed as –

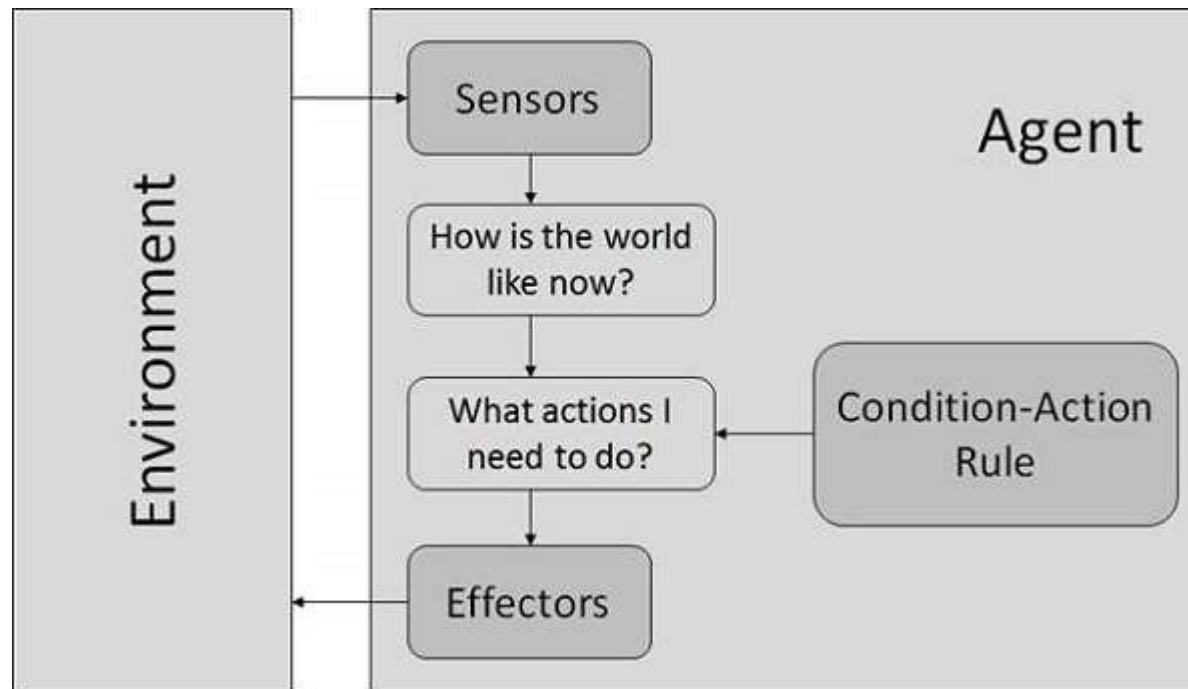
- Agent = Architecture & Agent Program
- Architecture = the machinery that an agent executes on.
- Agent Program = an implementation of an agent function.

Agents

1. Simple Reflex Agents

- They choose actions only based on the current percept.
- They are rational only if a correct decision is made only on the basis of current precept.
- Their environment is completely observable.

Condition-Action Rule – It is a rule that maps a state (condition) to an action.



Agents

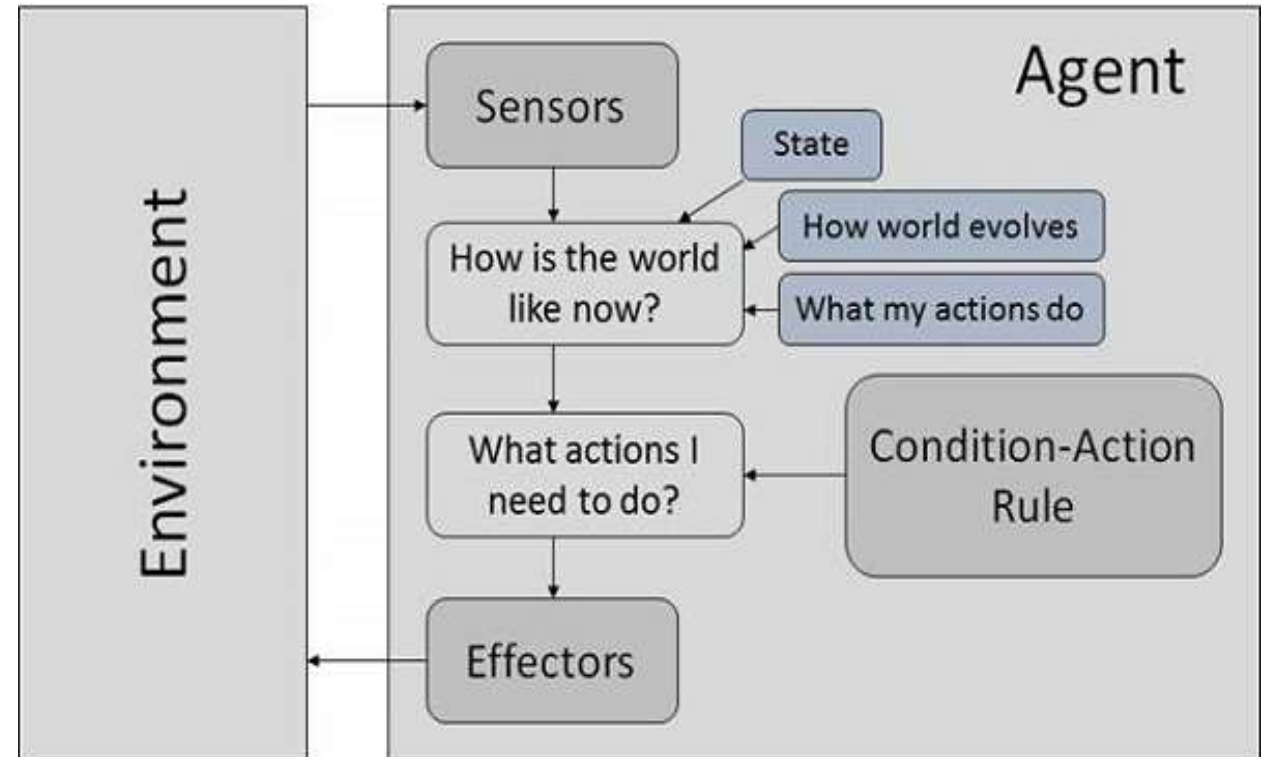
2. Model Based Reflex Agents

They use a model of the world to choose their actions. They maintain an internal state.

- Model – knowledge about how the things happen in the world.
- Internal State – It is a representation of unobserved aspects of current state depending on percept history.

Updating the state requires the information about –

- How the world evolves.
- How the agents actions affect the world.

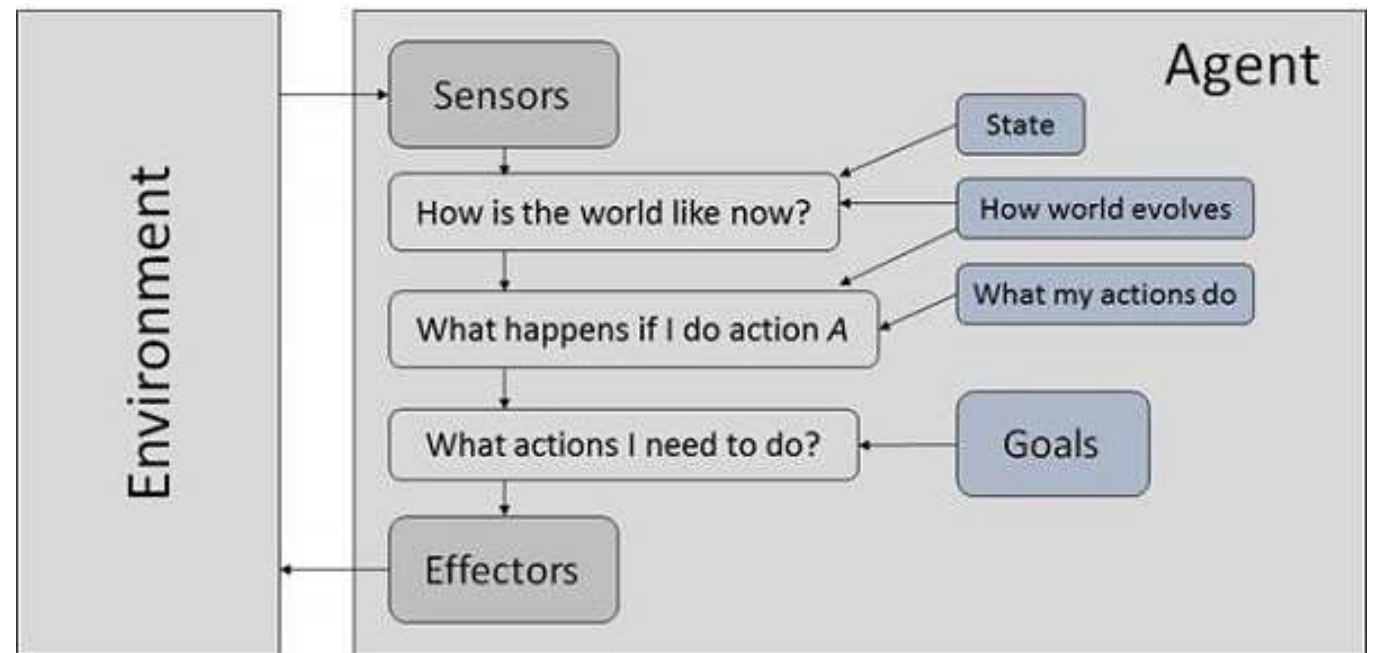


Agents

3. Goal Based Agents

- They choose their actions in order to achieve goals. Goal-based approach is more flexible than reflex agent since the knowledge supporting a decision is explicitly modeled, thereby allowing for modifications.

Goal – It is the description of desirable situations.



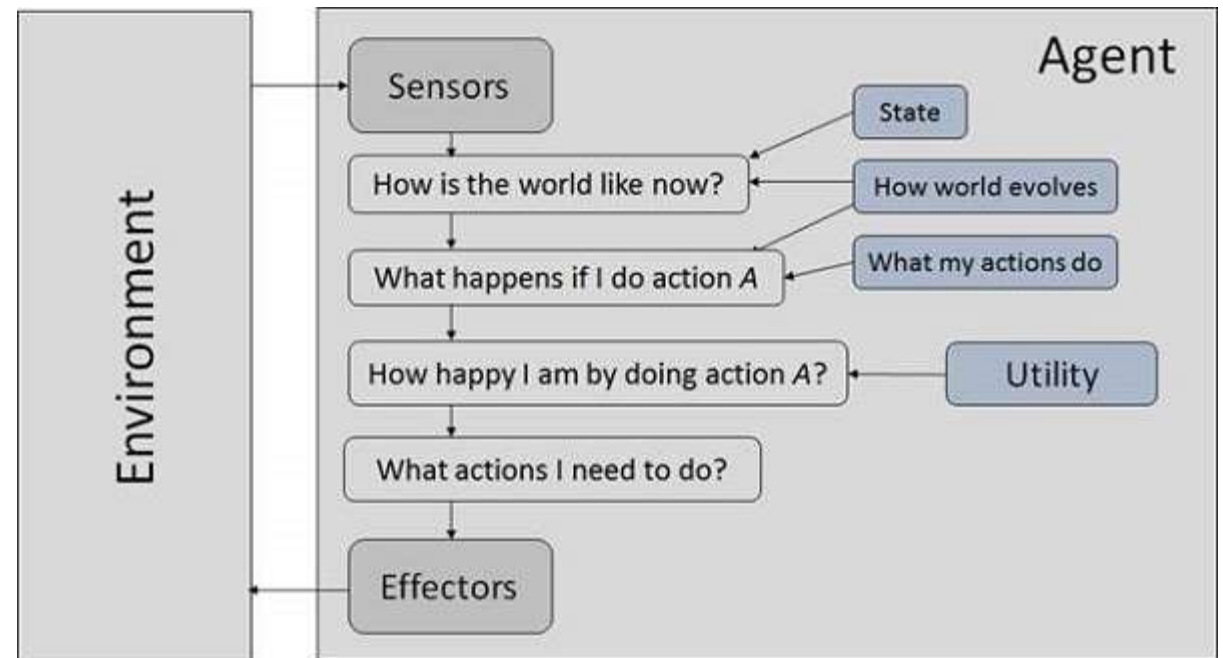
Agents

4. Utility Based Agents

- They choose actions based on a preference (utility) for each state.

Goals are inadequate when –

- There are conflicting goals, out of which only few can be achieved.
- Goals have some uncertainty of being achieved and you need to weigh likelihood of success against the importance of a goal.



The Nature of Environments

- Some programs operate in the entirely artificial environment confined to keyboard input, database, computer file systems and character output on a screen.
- In contrast, some software agents (software robots or softbots) exist in rich, unlimited softbots domains. The simulator has a very detailed, complex environment. The software agent needs to choose from a long array of actions in real time. A softbot designed to scan the online preferences of the customer and show interesting items to the customer works in the real as well as an artificial environment.
- The most famous artificial environment is the Turing Test environment, in which one real and other artificial agents are tested on equal ground. This is a very challenging environment as it is highly difficult for a software agent to perform as well as a human.

Properties of Environment

The environment has multifold properties –

1. **Discrete / Continuous** – If there are a limited number of distinct, clearly defined, states of the environment, the environment is discrete (For example, chess); otherwise it is continuous (For example, driving).
2. **Observable / Partially Observable** – If it is possible to determine the complete state of the environment at each time point from the percepts it is observable; otherwise it is only partially observable.
3. **Static / Dynamic** – If the environment does not change while an agent is acting, then it is static; otherwise it is dynamic.
4. **Single agent / Multiple agents** – The environment may contain other agents which may be of the same or different kind as that of the agent.
5. **Accessible / Inaccessible** – If the agents sensory apparatus can have access to the complete state of the environment, then the environment is accessible to that agent.
6. **Deterministic / Non-deterministic** – If the next state of the environment is completely determined by the current state and the actions of the agent, then the environment is deterministic; otherwise it is non-deterministic.
7. **Episodic / Non-episodic** – In an episodic environment, each episode consists of the agent perceiving and then acting. The quality of its action depends just on the episode itself. Subsequent episodes do not depend on the actions in the previous episodes. Episodic environments are much simpler because the agent does not need to think ahead.

Problem Solving in Artificial Intelligence

Problem Solving in Artificial Intelligence

- Problem-solving in Artificial Intelligence (AI) is the process of identifying a goal, analyzing available options, and selecting the most effective sequence of actions to achieve the desired outcome. It enables intelligent systems to reason, search for solutions, and make decisions in complex situations.
- Enables AI agents to make goal-oriented decisions in different environments.
- Forms the foundation of applications such as robotics, planning, navigation, and game playing.

The picture can't be displayed.

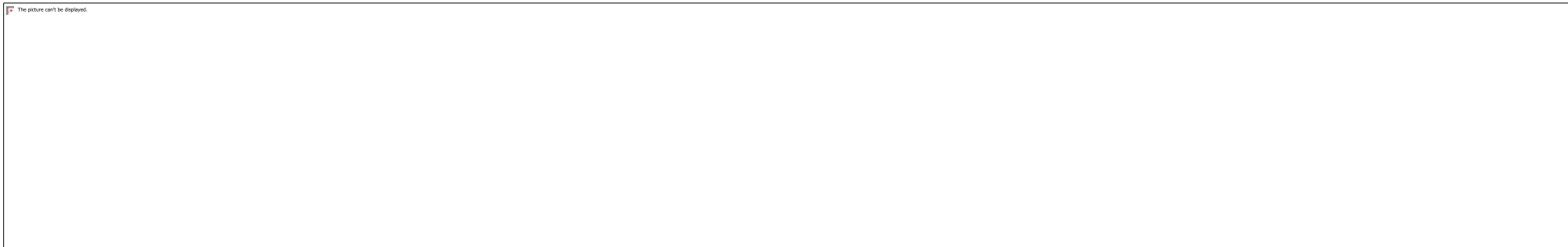
Problem Formulation in AI

Problem formulation is the process of defining a problem in a structured way so that an AI system can identify and achieve its goal efficiently.

- **Initial State:** Represents the starting point from which the AI agent begins solving the problem.
- **Actions:** Defines the possible operations or choices available to the agent at a given state.
- **Transition Model:** Describes how the state of the system changes after an action is performed.
- **Goal Test:** Determines whether the current state satisfies the desired goal.
- **Cost Function:** Measures the cost of reaching the goal and helps identify the most efficient solution path.

Working

- Goal Formulation: The AI identifies the objective it needs to achieve. A clearly defined goal helps the agent focus on the desired outcome and guides the problem-solving process.
- Problem Definition: The problem is clearly specified by identifying the initial state, goal state, constraints, and available actions. This establishes the framework for finding a solution.
- Defining the State Space: The AI represents all possible states and transitions that can occur while solving the problem. This helps the agent explore different paths toward the goal.
- Testing the Goal State: The system checks whether the current state satisfies the desired goal conditions. If the goal is reached, the search process can terminate.
- Calculating Cost of Path: The AI evaluates the cost associated with different solution paths, such as time, distance, or resources, and selects the most efficient path.



Techniques

- Several techniques are available to solve different types of problems in AI. The choice of technique depends on the nature of the problem and the resources available.



Techniques

1. Search Algorithms

Search Algorithms are important for exploring possible solutions. There are two main categories:

- Uninformed Search: These algorithms such as breadth-first and depth-first search, don't use additional knowledge about the problem and explore all possible paths.
- Informed Search: Algorithms like A* use heuristics or domain-specific knowledge to guide the search toward more promising paths making the search process more efficient.

2. Constraint Satisfaction Problems (CSP)

- In CSPs the goal is to find solutions that satisfy a set of constraints. Techniques like backtracking, constraint propagation and local search are used to explore the solution space efficiently while ensuring that all constraints are met.

Techniques

3. Optimization Techniques

AI often addresses optimization problems where the goal is to find the best possible solution among many options. Techniques include:

1. **Linear programming** for solving problems involving linear relationships.
2. **Dynamic programming** for breaking down complex problems into smaller subproblems.
3. **Evolutionary algorithms** like genetic algorithms that simulate natural selection to find optimal solutions.

4. Machine Learning

Machine learning allows AI systems to learn from data and improve over time. The three major learning types are:

1. **Supervised Learning:** Learning from labeled data to make predictions or classifications.
2. **Unsupervised Learning:** Discovering patterns in data without labeled outputs.
3. **Reinforcement Learning:** Learning through trial and error, receiving feedback in the form of rewards or penalties.

5. Natural Language Processing (NLP)

Natural Language Processing (NLP) helps AI systems understand and generate human language. Techniques like tokenization, Part-of-speech (PoS) and named entity recognition are important for chatbots, language translation and text summarization.

Applications, Advantages, Challenges

Applications

- Robotics and Navigation: Enables robots to navigate environments and perform tasks autonomously.
- Game Playing: Helps AI analyze strategies and make optimal decisions in games.
- Medical Diagnosis: Assists in identifying diseases by analyzing medical data and symptoms.
- Route Planning and Logistics: Finds efficient routes to reduce travel time and operational costs.
- Recommendation Systems: Suggests relevant products, services, or content based on user preferences.

Advantages

- Efficiency: Solves problems quickly by processing large amounts of data.
- Accuracy: Produces reliable results when trained with quality data.
- Automation: Reduces human effort by handling repetitive tasks automatically.
- Adaptability: Learns from new data and improves performance over time.
- Cost-Effectiveness: Increases productivity and reduces operational costs.

Challenges

- Complexity: Some problems require significant computational resources.
- Data Quality: Poor or biased data can affect decision accuracy.
- Interpretability: Advanced AI models can be difficult to understand and explain.
- Ethics and Bias: AI systems may inherit biases present in training data.

Formulation of problems:
Vacuum world, 8 queens,
Route finding-robot navigation

1. Vacuum world

- Intelligent Agents takes a vacuum-cleaner world to describe the concept of agent function, agent program, and agent designs.
- The environment consists of locations (e.g., Rooms A and B) that can be clean or dirty, with an agent located in one of them.
 - **Initial State:** Any room can be dirty or clean, and the agent can start in either location.
 - **Actions:** {Left, Right, Sweep}.
 - **Transition Model:** Predictable; moving left in the leftmost room does nothing, and sucking in a clean room has no effect.
 - **Goal Test:** Checks if all the squares in the environment are clean.
 - **Path Cost:** Each action typically incurs a cost of $\backslash(1\backslash)$ (e.g., measuring power/energy or time consumed).

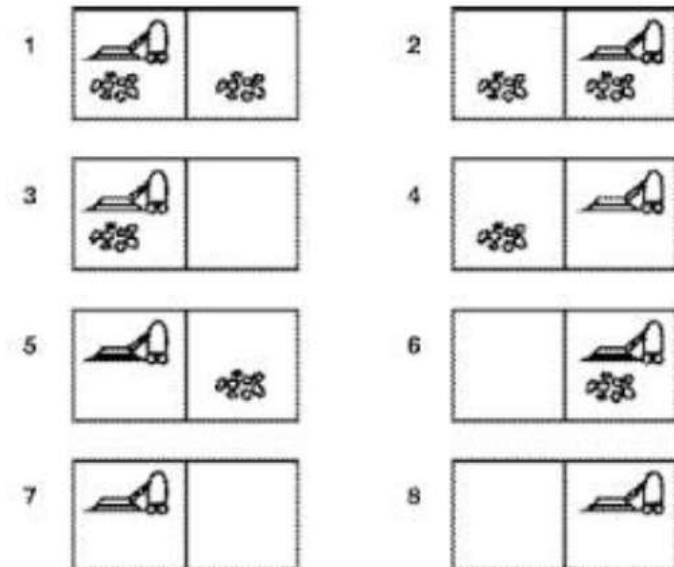
Vacuum Cleaner Problem in Artificial Intelligence

In the book Artificial Intelligence: A Modern Approach, Chapter 2 (Intelligent Agents) takes a vacuum-cleaner world to describe the concept of agent function, agent program, and agent designs.

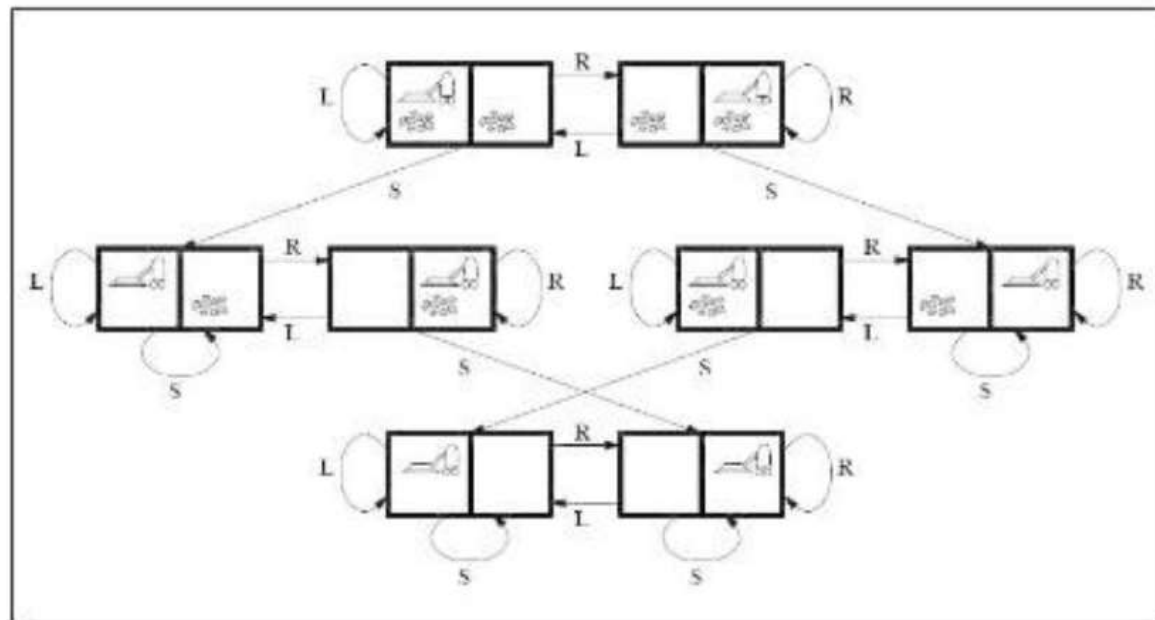
In the simple world, the vacuum cleaner agent has a **location sensor** and a **dirt sensor** so that it knows where it is (room A or room B) and whether the room is dirty. It can go left, go right, suck, and idle. A possible performance measure is to maximize the number of clean rooms over a certain period. Each step has an associated cost, be it in power/ energy consumption, noise created, or wear and tear. An efficient and rational agent would maximize performance while mitigating all costs possible. If an agent cleans one square, cleans another, and then cleans the previously cleaned square again, it would be wasting time and power, as well as increasing time and wear & tear. However, if it can efficiently track cleaned and dirty squares, and work out a way of ideally visiting a square only once, this would result in the best-case scenario, mitigated time steps, that would translate into time, noise, power and depreciation savings.

Consider the following Vacuum cleaner world:

Let's suppose that the world has just two rooms. The robot can be in either room and there can be dirt in zero, one, or two rooms.



State Space Graph:



Percept sequence	Action
$[A, Clean]$	<i>Right</i>
$[A, Dirty]$	<i>Suck</i>
$[B, Clean]$	<i>Left</i>
$[B, Dirty]$	<i>Suck</i>
$[A, Clean], [A, Clean]$	<i>Right</i>
$[A, Clean], [A, Dirty]$	<i>Suck</i>
$\vdots$	$\vdots$

- Which type of agents would be appropriate for this problem?

2. 8-Queens Problem

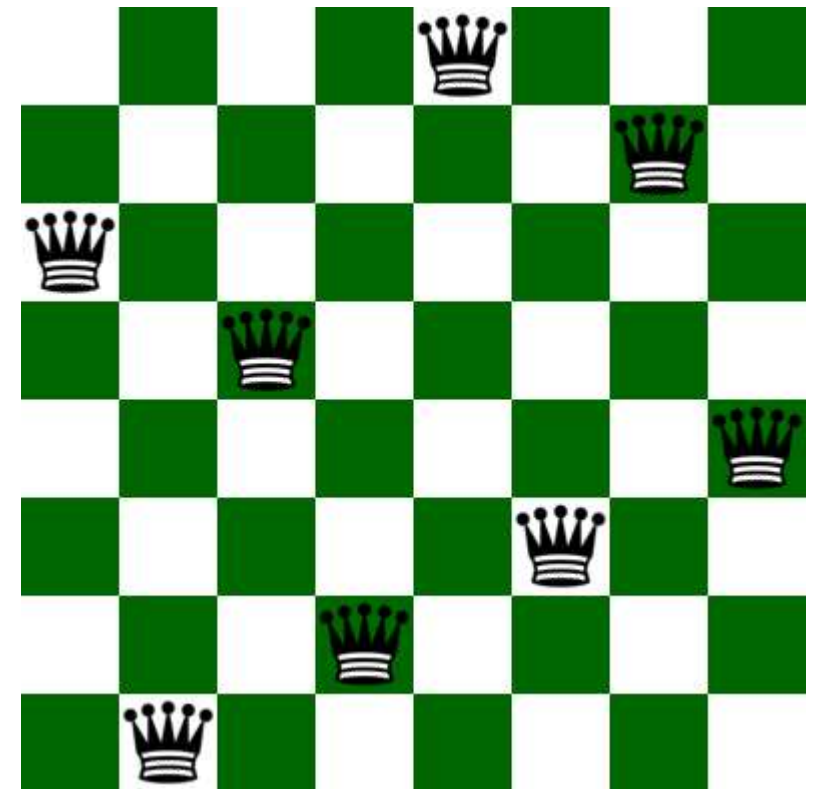
- The goal is to place 8 queens on an 8 X 8 chessboard such that no two queens threaten each other (i.e., no two are in the same row, column, or diagonal).
 - **Initial State:** An empty 8 X 8 board.
 - **Actions:** Add a new queen to any square in the next empty row that is not attacked by any existing queen. This focuses the search and reduces the state space compared to placing a queen anywhere on the board.
 - **Transition Model:** The board configuration updates as a queen is successfully added.
 - **Goal Test:** A condition that checks if all 8 queens are on the board and none are attacking each other.
 - **Path Cost:** Often path cost is 0 because we only care about the primary objective i.e. finding the final state

State Space Search

- Given an 8x8 chessboard, the task is to place 8 queens on the board such that no 2 queens threaten each other. Return a matrix of size 8x8, where 1 represents queen and 0 represents an empty position.
- Algorithms like [Backtracking Search](#) are a fundamental tool for such constraint satisfaction problems, as they efficiently navigate the [state space](#) by abandoning paths that fail the constraints early.
- When approaching the 8-Queens problem in AI, the [choice of search algorithm is critical](#). Several algorithms can be used, each with its advantages and trade-offs. Here, we discuss [three](#) common choices:

1. Backtracking Search:

- This is the [most common and intuitive](#) algorithm for the 8-Queens problem.
- It places queens one by one, row by row. If it cannot find a valid spot for a queen in the current row, it [backtracks](#) to the previous row and moves that queen.
- It is a [form of Depth-First Search \(DFS\)](#) with pruning, making it highly memory-efficient.



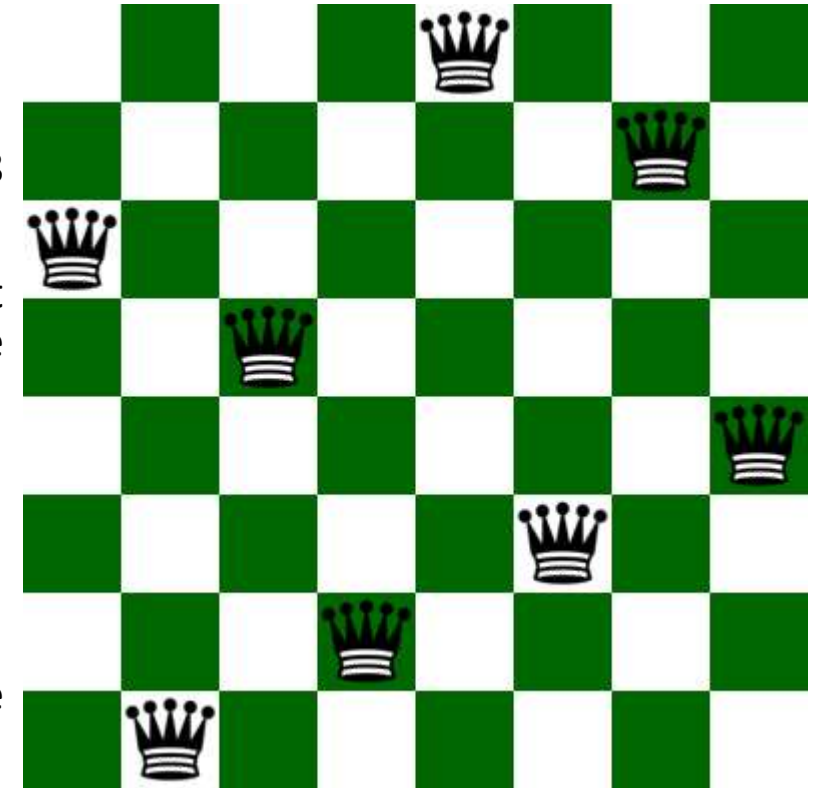
State Space Search

2. Depth-First Search (DFS):

- A standard, **uninformed DFS** would explore all possible arrangements of 8 queens on the board.
- It is **less efficient than backtracking** for this problem because it does not prune invalid states early (e.g., boards where two queens are already in the same column).

3. Constraint Satisfaction Problem (CSP) Solvers:

- The search tree is a fundamental concept in solving the 8-Queens problem.
- The root node is the empty board.
- Edges from a node represent placing a new queen in a valid column of the next row.
- The tree's depth corresponds to the number of queens placed.
- Leaf nodes at depth 8 represent complete assignments (potential solutions).



Search algorithms like **backtracking** systematically expand this tree but **prune branches** where a queen cannot be legally placed, drastically reducing the number of states explored.

3. Route finding-robot navigation problems

In AI, route finding and robot navigation are formulated as state-space search problems. They require an agent to map a physical environment into states, identify allowed movements (actions), and apply search algorithms (like A^* , Dijkstra, or RRT) to determine a safe, cost-effective path to a target.

Formulating these problems requires defining five fundamental components:

- **Initial State:** The agent's or robot's starting location and orientation.
- **State Space:** The set of all possible positions in the environment, which can be discrete (a grid or graph) or continuous (real-valued coordinates).
- **Actions:** The possible movements (e.g., move forward, turn, stop) to transition from one state to another.
- **Transition Model:** The rules determining exactly what state results from taking a specific action. Obstacles or inaccessible regions are factored into these transitions.
- **Path/Action Cost:** A numerical value that penalizes certain routes, such as total distance, energy consumption, or time taken.

3. Route finding-robot navigation problems

The Core Problem in Robotics: Configuration Space (C-Space)

To avoid having the robot collide with objects, AI models frequently transform robot navigation into Configuration Space (C-Space). Instead of planning a complex shape that rotates and translates, the robot is simplified to a single "point" while the obstacles are grown/expanded to compensate for the robot's physical dimensions. The AI then computes the most efficient route for this point across the free-space map.

Popular AI Algorithms Used Depending on the environment, engineers deploy specific AI algorithms to solve the formulated navigation problem:

- **Classical Graph Search (A*, Dijkstra):** These find the absolute shortest route but require static, well-known environments.
- **Sampling-Based Planners (RRT, PRM):** Used for complex, higher-dimensional spaces. Instead of checking every possible location, algorithms randomly sample points to build a tree of feasible paths.
- **Dynamic Motion Planners (DWA, TEB):** Used to avoid moving obstacles in real-time, focusing on local adjustments.
- **Machine Learning (Reinforcement Learning):** Used to adaptively learn optimal, safe trajectories by trial and error, particularly for navigating dynamic or entirely unknown environments.

End of Unit - 1